

Centralized Task Planning and Dynamic Device Collaboration in LLM-based Multi-Agent Smart Home Systems

Zeyi Sun¹, Jinghua Groppe^{1,2}, and Sven Groppe^{1,2}

¹ University of Lübeck, Germany

² Institute of Computer Science, TU Bergakademie Freiberg, Freiberg, Germany,
zeyi.sun@student.uni-luebeck.de
{jinghua.groppe,sven.groppe}@uni-luebeck.de

Abstract. Coordinating multiple smart home devices under complex natural language instructions remains challenging for traditional rule-based systems. While recent LLM-based agentic approaches enable autonomous reasoning, many existing designs rely on centralized orchestration and proprietary large-scale models, limiting scalability and cost efficiency. To address these challenges, we introduce an LLM-based multi-agent smart home system that combines automatic task decomposition with adaptive device-level collaboration and built upon open-source models. Our system can automatically generate a structured execution plan from user input and allows device agents to collaborate dynamically during runtime. To address the challenge of systematic evaluation of LLM-based multi-agent systems in smart home, we also develop a reusable and system-agnostic benchmark comprising 60 categorized smart home scenarios with different complexities. The experimental evaluation on the benchmark demonstrates our LLM-based multi-agent smart home can carry out effect coordination in low- and moderate-complexity scenarios, but has a performance degradation under high complex scenarios indicating that coordination across a vast array of intentions remains a significant challenge. The benchmark and evaluation framework have been open-sourced at <https://github.com/katzezwii/smart-home-llm-multi-agent/tree/main/benchmark>.

Keywords: Agentic AI · LLM-based agents · Multi-agent systems · Smart home coordination · Benchmarking

1 Introduction

Smart homes today consist of many interconnected IoT devices, such as lighting systems, audio systems, calendars, and search services [2,15]. Although device connectivity has improved significantly, coordinating multiple devices under complex natural language instructions remains difficult. Traditional rule-based automation systems rely on predefined trigger-action rules and fixed workflows [19,13]. These systems work well for simple commands such as “turn on the

kitchen light”, but they fail when users express multiple intentions or provide implicit context, such as “I just got home and feel tired, make it relaxing”.

Recent advances in agentic AI and Large Language Models (LLMs) have introduced a new paradigm: LLM-based agentic systems. An LLM-based agentic AI system refers to an autonomous system capable of pursuing complex, goal-oriented tasks through contextual reasoning, multi-step planning, tool usage and adaptive decision-making [20]. Unlike traditional task-specific automation, such systems emphasize autonomy and adaptability in dynamic environments, making them particularly suitable for smart home coordination [1].

Several efforts have worked on LLM-based agent systems for smart home [10,16]. In these approaches, a single high-level LLM-based agent handles both planning and device coordination. Although these systems exhibit robust reasoning capabilities, they typically depend on access to powerful (often proprietary) LLMs and involve non-trivial prompt engineering and control structures. Furthermore, as the device ecosystem expands, the computational overhead and context window constraints of a centralized agent may impede scalability and system efficiency. Conversely, fully decentralized architectures—wherein individual devices or local agents independently interpret and act on user input—can lead to challenges in maintaining consistency, avoiding redundant computation, and ensuring effective coordination across devices. Without mechanisms for shared global state or task decomposition, such systems risk producing incoherent or conflicting behaviors.

To address these limitations, we propose an LLM-based Multi-Agent System (LLM-MAS), which integrates centralized task planning with dynamic device collaboration. LLM-MAS architectures represent the evolution of Agentic AI from a “solo worker” model to a “team collaboration” model. Instead of a single AI agent trying to handle every aspect of a complex task, multiple specialized AI agents work together, interacting and collaborating to solve problems that are too complex for a single agent [11,4]. The transformative potential of LLM-MAS has been applied e.g. to address complex challenges in software engineering [8], to automate the archival descriptions [6], to design instructional guidelines for courses and learning activities [22], to optimize the transaction scheduling problem [24], and to improve user experience and data quality on citizen science platforms [9].

One of aims of this work is exploring the transformative potential of LLM-MAS in addressing challenges in developing smart home. In our LLM-MAS, a centralized task planning layer generates a structured task queue to provide global execution structure, while device agents execute tasks and initiate peer collaboration when necessary. This design provides global task structuring without enforcing centralized control, thereby preserving decentralized agent autonomy. Furthermore, We propose a constrained dynamic collaboration mechanism in which device agents autonomously decide whether to initiate single-round peer interactions. This mechanism is ‘constrained’ by limiting collaboration to a single exchange between a requesting agent A and one target agent B. This structural limit eliminates the risk of cyclic collaboration loops, thereby ensuring

system stability. Furthermore, the design guarantees predictability by bounding execution paths to two scenarios: an agent either completes the task directly or follows a defined interaction sequence ($A \rightarrow B \rightarrow A \rightarrow \text{Task Planner}$) before returning to the Task Planner.

Another aim of this work is providing a solution in the systematic evaluation of LLM-MAS based smart home solutions. Recent surveys highlight that evaluation of agentic systems is fragmented and often domain-specific, typically measuring dimensions such as task completion, planning capability, and reliability [21,14]. However, structured benchmarks for end-to-end smart home orchestration remain limited. To support systematic and reproducible assessment in this domain, we introduce a reusable and system-agnostic benchmark consisting of 60 categorized smart home scenarios. The benchmark evaluates task planning quality, collaboration behavior, and overall task completion performance across varying levels of intent.

This work contains the following main contributions:

- An LLM-based multi-agent architecture featuring centralized task planning and dynamic device-level collaboration, which can maintain decision consistency, avoid redundant computation, and ensure effective coordination across devices.
- A constrained dynamic collaboration mechanism in which device agents autonomously decide whether to initiate single-round peer interaction, ensuring stable and predictable coordination.
- A system-agnostic benchmark with varying levels of task complexity for evaluating LLM-MAS-based smart home systems³
- An extensive experimental evaluation for end-to-end smart home systems using the benchmark tests.
- A systematic and comprehensive analysis and discussion of the evaluation results.

2 Related work

Traditional Smart Home Architectures. Traditional smart home systems primarily rely on rule-based automation and predefined workflows. Their functionality is centered on device connectivity and the execution of manually configured routines [13]. While effective for handling explicit commands, such systems lack semantic reasoning capabilities and struggle with vague user requests. The dependence on user-defined rules further limits adaptability, preventing these architectures from dynamically inferring user intent or coordinating devices under complex scenarios [7,10]. These limitations have motivated the exploration of LLM-based approaches for more flexible and context-aware smart home automation.

³ The benchmark for evaluating LLM-MAS smart home is available at <https://github.com/katzezwei/smart-home-llm-multi-agent/tree/main/benchmark>.

LLM-Based Agentic Systems for Smart Homes. LLMs have recently been extended into autonomous agents capable of reasoning, task decomposition, and tool invocation [20,17,18,5]. By integrating external tools and APIs, LLM-based agents can translate natural language goals into executable actions. Recent research further explores the integration of LLM agents with Internet of Things (IoT) environments, enabling natural language control and autonomous coordination of connected devices [3]. In smart home environments, several systems leverage LLM agents within multi-agent architectures but retain centralized orchestration mechanisms. For example, systems such as SAGE and SASHA employ high-level reasoning components to coordinate device actions and manage execution flows [16,10]. While such designs improve reasoning coherence and control stability, they concentrate coordination decisions in a single component and may introduce scalability limitations. In particular, SASHA generates complete action plans that are executed sequentially, while SAGE relies on a centralized LLM orchestrator for dynamic decision-making. Both approaches concentrate reasoning, planning, and execution within a single component. This observation motivates the exploration of architectures that separate task planning from execution, allowing device agents to independently determine how tasks should be carried out and whether collaboration is required.

Evaluation and Benchmarking of LLM-Based Agents. Evaluating LLM-based agents has recently become an important research topic. Recent surveys highlight that evaluation methodologies for LLM agents remain fragmented and often domain-specific, typically measuring capabilities such as task completion, planning ability, and reliability [21,14]. To support systematic evaluation, several benchmarks have been proposed for assessing agent performance in interactive environments. For instance, AgentBench provides a comprehensive framework for evaluating LLMs acting as autonomous agents across diverse tasks and environments [12]. More recently, MultiAgentBench investigates the evaluation of collaborative behaviors among multiple agents in complex scenarios [23]. However, existing benchmarks primarily focus on general-purpose agent capabilities or abstract multi-agent collaboration. Structured evaluation frameworks targeting domain-specific multi-device orchestration scenarios, such as smart home environments, remain limited. This highlights the need for structured evaluation frameworks that examine coordination and task execution in domain-specific multi-device environments such as smart homes.

3 LLM-based Multi-Agent System for Smart Home

To address the limitations of traditional rule-based systems and current agent approaches, we developed an LLM-based multi-agent smart home system. This system integrates automatic task decomposition with adaptive device-level coordination mechanisms, decoupling task organization from device-level execution while enabling controllable collaboration among agents. The following subsections describe the system architecture, execution workflow, task planning process, device agent design, and the collaboration mechanism.

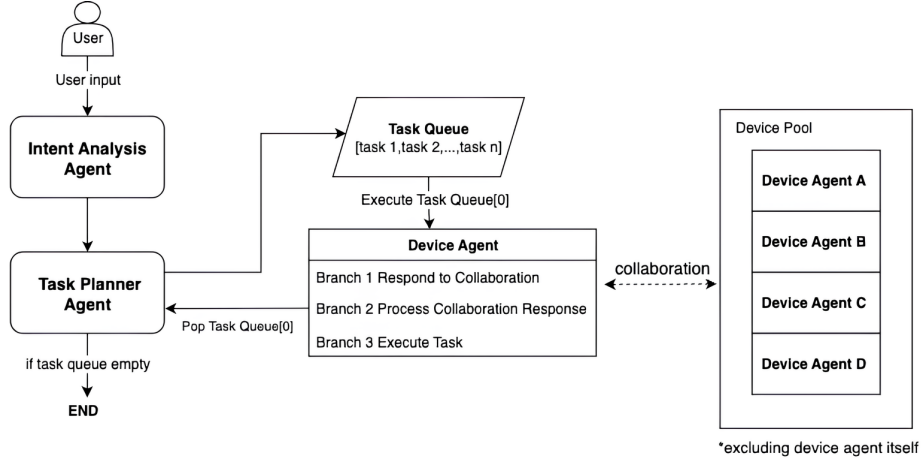


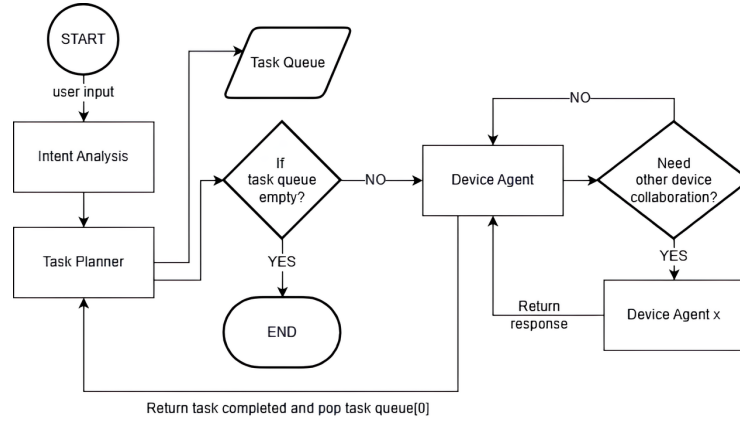
Fig. 1. Architecture of the LLM-based multi-agent smart home system. User input is analyzed and decomposed into a task queue, which is executed by LLM-based device agents with optional peer-to-peer collaboration.

3.1 System Architecture

The proposed architecture combines centralized task planning with decentralized device-level execution. As illustrated in Fig. 1, the system consists of four main components: a user input layer, an Intent Analysis Agent, a Task Planner Agent, and a pool of LLM-based Device Agents. User input is first processed within a centralized planning layer composed of an Intent Analysis Agent and a Task Planner Agent. The Intent Analysis Agent evaluates the complexity of the input by identifying independent information points and extracting relevant keywords. Based on this analysis, the Task Planner constructs a task queue consisting of device–task pairs that represent the actions required to fulfill the user request. Device Agents are domain-specialized LLM-based agents responsible for executing assigned tasks. Each agent performs local reasoning to interpret task requirements and may initiate peer-to-peer collaboration with other agents when additional information is required. This dynamic collaboration mechanism enables agents to coordinate during execution while maintaining a controlled single-round interaction structure.

3.2 System Workflow

Figure 2 depicts the execution workflow of the system. After receiving user input, the system performs intent analysis, task decomposition, and sequential task execution through device agents, while allowing agents to request collaboration when necessary. To better explain this process, we consider the *complex_19* case from the benchmark with the following user request:

**Fig. 2.** System Workflow

“The weather’s awful today. Check the outdoor temperature for me, then set the indoor temperature a bit warmer than outside.”

First, the user request is processed by the Intent Analysis Agent and the Task Planner Agent, which together generate a structured task queue. For this example, the resulting task queue contains two tasks: (1) the search engine agent checks the outdoor temperature, and (2) the thermostat agent sets the indoor temperature slightly warmer than the outdoor temperature. The system then executes the task queue sequentially, starting from the task at the top of the queue. The first task is assigned to the search engine agent, which retrieves the outdoor temperature (e.g., 15°C in Hamburg). After completing this task, it is removed from the queue and the system proceeds to the next task. The thermostat agent then uses the retrieved temperature, which is available through the shared interaction history among agents. This information reuse represents an implicit form of collaboration, as further discussed in Section 3.5. Based on this information, the agent determines an appropriate indoor temperature and executes the action, e.g., setting the indoor temperature to 18°C. The process continues until the task queue becomes empty, indicating that the user request has been fully completed.

3.3 Task Planning

Within the task planning layer, user input is first processed by the Intent Analysis Agent to extract information points and key modifiers that may guide subsequent task decomposition. These extracted elements do not directly determine task execution but instead provide structured hints for the Task Planner Agent.

For instance, in the *simple_5* test case in the benchmark with the user input (*“I’m going to study.”*), the agent identifies a single information point (*“I’m going to study”*) without any key modifier. In contrast, the *moderate_02* test case (*“I need to study for 45 minutes, brighten the light and play some focus music.”*)

yields three information points: “I need to study for 45 minutes,” “brighten the light,” and “play some focus music.” The corresponding key modifiers include “for 45 minutes,” “brighten,” and “some focus music.” These extracted elements as well as the user input are provided to the Task Planner Agent as contextual hints to support task decomposition.

Consider again the benchmark test case introduced in Section 3.2. From this input, the Intent Analysis Agent identifies two information points: retrieving the outdoor temperature and adjusting the indoor thermostat. In addition, the phrase “a bit warmer” is extracted as a key modifier. Based on the current system state, the Task Planner Agent is then invoked to determine the next step. Since the task queue is initially empty while the original user input exists, the planner follows the *Task Queue Generation Branch*. In this step, the planner generates a structured task queue consisting of two high-level tasks: retrieving the outdoor temperature using the search engine agent and adjusting the indoor thermostat accordingly.

The Task Planner Agent does not directly specify how each device should perform the task. Instead, it produces high-level device-oriented tasks and leaves execution decisions to the corresponding device agents. During execution, the planner acts as a routing component that dispatches the task at the top of the task queue to the appropriate device agent. Once a task is completed, it is removed from the queue and the planner is invoked again. This process continues until the task queue becomes empty. At that point, since the original input has already been consumed during task generation, the planner follows the *All Tasks Completion Branch*, indicating that the entire user request has been successfully processed.

3.4 Device Agent Abstraction

Each device is implemented as a stateful, domain-specialized LLM-based agent responsible for task execution and optional collaboration. While semantic reasoning is performed by the LLM, execution follows a predefined conditional branching structure. Each device agent processes incoming state variables according to three ordered conditions: (1) respond to incoming collaboration requests (Collaboration Request Branch), (2) resume tasks awaiting collaboration results (Pending Task Branch), and (3) execute newly assigned tasks from the task queue (New Task Branch).

To illustrate this process, consider the example introduced in Section 3.2. When the task planner routes the first task (“check the outdoor temperature”) to the search engine agent, the agent executes the task through the New Task Branch and stores the retrieved temperature in the shared interaction history. When the second task (“set the indoor temperature a bit warmer than outside”) is routed to the thermostat agent, the agent again follows the New Task Branch. During reasoning, it accesses the previously retrieved temperature from the shared interaction history and determines an appropriate indoor temperature. In this example, the thermostat sets the indoor temperature to 18°C, warmer than

the outdoor temperature of 15°C. Because the required information is already available in the shared history, no explicit collaboration request is needed.

This design ensures that only one branch is executed during each agent invocation, providing a clear and deterministic control flow. Prioritizing collaboration requests also enables agents to respond promptly to requests from other agents. Fig. 3 illustrates an execution path involving explicit collaboration. In this example, the lighting agent issues a collaboration request to the clock agent to set a timer for 10 pm. The clock agent handles the request through the Collaboration Request Branch. After receiving the response, the lighting agent resumes the pending task via the Pending Task Branch and completes the lighting schedule.

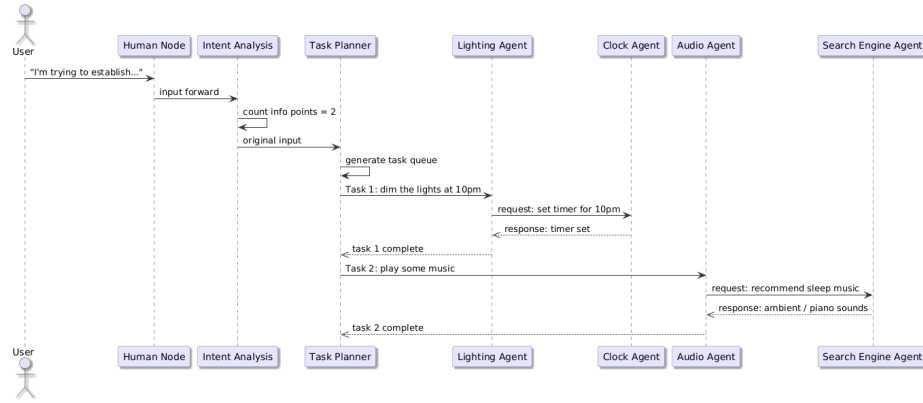


Fig. 3. Sequence diagram of the bedtime routine, where the user says: “I’m trying to establish a bedtime routine. I want the lights to dim gradually starting at 10pm. Also play some music to help me sleep.”

3.5 Dynamic Collaboration Mechanism

Device agents may initiate collaboration when the LLM determines that additional information is required to complete a task. In such cases, the agent generates a collaboration request specifying a single target device and temporarily transfers control to that agent. The interaction follows a single-round structure: the target agent responds to the request and immediately returns control to the requester. The responding agent does not initiate further collaboration requests, and the original agent resumes and completes its pending task after receiving the response. After the Device Agent completed task, control returns to the Task Planner, which continues routing the remaining tasks in the queue.

Collaboration in the system can occur in two forms: implicit collaboration and explicit collaboration. Implicit collaboration arises when agents reuse information stored in the shared interaction history without issuing a direct request. The example discussed in Section 3.2 illustrates this case. In contrast, explicit

collaboration occurs when an agent actively sends a collaboration request to another agent. Fig. 3 illustrates an explicit collaboration scenario in which the lighting agent requests the clock agent to set a timer for the dimming action at 10pm. After receiving the response, the lighting agent resumes and completes the task.

4 Benchmark

Table 1. Device-agent capabilities

Device	Primary Capabilities
clock	Set alarm or timer
calendar	Schedule, event, reminder
search engine	Information lookup
tv display	Display visual content
fridge	Food inventory
lighting	Illumination control
thermostat	Temperature control
audio system	Sound/volume control

Table 2. Intent Complexity distribution of the benchmark test cases

Complexity (#intents)	#cases
Simple (1-2)	20
Moderate (3)	20
Complex (4+)	20
Total	60

Evaluating LLM-based multi-agent systems requires structured benchmarks that assess coordination and reasoning across diverse scenarios. Existing benchmarks primarily focus on general-purpose domains such as research collaboration or web navigation [23,14], but lack domain-specific evaluation frameworks for smart home environments. To address this gap, we develop a benchmark for evaluating LLM-based multi-agent smart home systems. Due to the limitation of pages, we give here an overview of the benchmark and the complete benchmark dataset is available in the project repository (<https://github.com/katzezweii/smart-home-llm-multi-agent/tree/main/benchmark>).

Our domain-specific benchmark is designed to evaluate intent recognition, collaboration correctness, and overall task completion of Agentic AI smart home systems. It comprises 60 realistic smart home scenarios, eight abstract device types and three categories of intents. Table 1 outlines the 8 device types and each device is an autonomous agent. Among the intents, *Required* intents represent explicit user requests where specific actions must be identified. *Acceptable* intents represent ambiguous requests where multiple interpretations are possible, but the system must identify at least one of the listed intents to be considered correct. *Mixed* specifications combine both cases, requiring explicit intents while allowing additional acceptable intents.

The 60 text cases are divided into three complexity levels based on the complexity of intents and the number of participating devices. Table 3 reports the average number of participating devices and device range for each complexity level. As expected, the average number of participating devices increases

Table 3. Intent complexity and the average and range of device counts

Level	Intent Count	Avg. #device	Range #device
Simple	1–2	2.05	1–5
Moderate	3	3.20	2–5
Complex	4+	4.05	2–7
Total	1–8	3.10	1–7

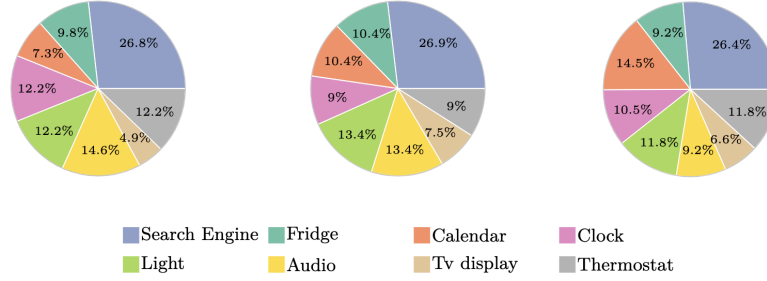


Fig. 4. Device distribution across intent complexity groups. From left to right: simple (1–2 intents), moderate (3 intents), and complex (4+ intents). The distribution shows consistent device coverage across groups, with Search Engine being the most frequently invoked device while other devices maintain balanced participation.

with intent complexity, indicating stronger coordination requirements in more complex scenarios. Figure 4 illustrates device distribution across complexity groups. Search Engine remains the most frequently invoked device (approximately 26–27% across all groups), while other devices exhibit relatively balanced participation. The distribution demonstrates broad device coverage and avoids dominance by any single device type.

Listing 1.1. A text case of simple level in the benchmark

```

{
  "id": "simple_01",
  "category": "simple",
  "user_input": "Help me find an Italian pasta recipe and see what
                 ingredients I'm missing",
  "required_intents": [
    {"intent": "Find Italian pasta recipe", "device_type": "search"},
    {"intent": "Identify missing ingredients", "device_type": "fridge"}
  ],
  "collaboration": {
    "is_needed": true,
    "success_conditions": [
      "Fridge obtains recipe information from search device"
    ]
  },
  "expected_final_output": "The system explicitly lists the missing
                           ingredients to the user"
}

```

Listing 1.1 presents a test case with a simple complexity level in the benchmark test.

5 Evaluation

Our LLM-based multi-agent smart home system is implemented using LangGraph (v1.0.5) built on LangChain (v1.2.0), and uses the open-source LLM *google/gemma-2-9b* to create a more cost-efficient system in comparison with commercial LLMs. The LLM is deployed locally via Ollama, with temperature set to 0.0 to ensure deterministic outputs. We carry out an extensive experimental evaluation of the smart home system on the benchmark developed on Sec. 4 using the evaluation criterion defined in Sec. 5.1

5.1 Evaluation Criteria

Each benchmark scenario is evaluated according to the following three dimensions:

- **Task Generation Quality.** This dimension evaluates whether the task planner correctly interprets user intents and decomposes them into appropriate device-level tasks.
- **Collaboration Success.** For scenarios involving multiple devices, this dimension measures whether agents correctly identify collaboration needs and exchange information to support task execution.
- **Overall Completion.** Overall completion evaluates the final outcome of the scenario by considering task generation correctness, collaboration outcomes, and the final device outputs produced by the agents. A scenario is successful only when the resulting device actions satisfy the original user request.

Each scenario is rated as *success* (all requirements are processed correctly), *partial* (Some of requirements are preprocessed wrongly), or *failure* (None of the requirements are processed correctly). Detailed evaluation rules are provided in the accompanying repository.

5.2 Performance by Intent Complexity

Intent complexity is defined by the number of intents the system must resolve from the input. In practice, higher complexity cases often appear as longer sentences containing multiple clauses or conditions. We manually evaluated the experimental data according to the three dimensions defined in Section 5.1 (task generation quality, collaboration success, and overall completion). Based on these criteria, each case was labeled as success, partial, or failure, and the results were reported in Table 4 and Fig. 5

Table 4 summarizes performance across intent complexity levels. Our LLM-MAS smart home archived good performance for simple cases: 16 out of 20 were processed completely correctly, 2 were partially correct, and 2 were incorrect. With the increasing complexity, the complete success rates also decrease accordingly. However, the number of full failure is still low (2 and 4 for the moderate and high complexity levels, which are similar to the simple cases). This suggests

that the system often correctly handles subsets of multi-intent requests even when full coordination fails. The increase in full failure rate from 10% (simple) to 20% (complex) indicates that handling multiple intents is significantly more difficult than executing simple intent tasks. Overall, the system achieved a 56.7% complete success rate across all 60 cases, with 30% partial completion and 13.3% full failures. Figure 5 demonstrates these trends.

Table 4. Performance by intent complexity

Level (#required-intents)	Cases	Success	Partial	Failure
Simple (1–2)	20	16 (80%)	2 (10%)	2 (10%)
Moderate (3)	20	11 (55%)	7 (35%)	2 (10%)
Complex (4+)	20	7 (35%)	9 (45%)	4 (20%)
Total	60	34 (56.7%)	18 (30%)	8 (13.3%)

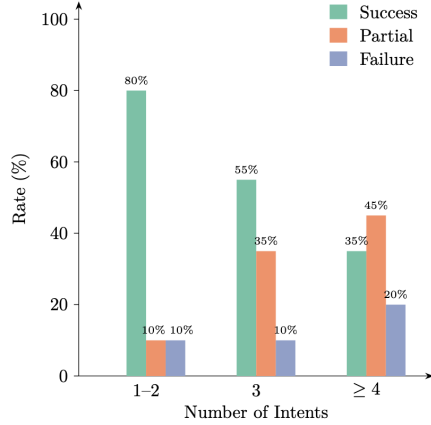


Fig. 5. Performance by Number of Intents.

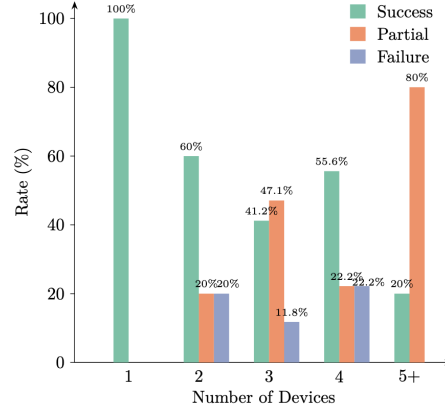


Fig. 6. Performance by Number of Devices.

5.3 Performance by Device Complexity

In addition to intent complexity, we further analyze the system performance in related to the device complexity, which is defined by the number of distinct device agents involved in executing a request. Higher device complexity indicates that more devices must be coordinated to fulfill the user request. Figure 6 presents performance trends as device count increases. Single-device tasks achieved 100% full success, showing that the system performs reliably when only one device

Table 5. Average response time by intent complexity

Group	Avg Response Time (seconds)
Simple (1-2)	39.91
Moderate (3)	57.19
Complex (4+)	78.18

is involved. The full success rate declines as the number of devices increases: 60% for two devices, 41.2% for three devices, and 20% for five or more devices. However, our smart home performed well on partial completions for high device complexity. For scenarios involving five or more devices, partial success reaches 80%. This indicates that our system can correctly handle subsets of multi-devices requests even when full coordination fails. The decrease in full success rates from 100% (simple) to 20% (complex) also suggests that coordinating multiple devices is a big challenge of in LLM-MAS systems.

5.4 Response Time Analysis

Table 5 reports average response times by intent complexity. Execution time increases substantially with complexity: 39.91s (simple), 57.19s (moderate), and 78.18s (complex). The increase in latency is primarily due to the larger number of involved devices and the sequential execution of agent interactions. Complex scenarios require more device invocations and may include inter-device collaboration steps, which accumulate over the execution pipeline and result in longer response times.

5.5 Case Analysis

To better understand system behavior, we analyze several representative examples of successful, partially successful, and failed executions.

Successful Case. The test case `complex_19` in the benchmark with the user request “*The weather’s awful today. Check the outdoor temperature and set the indoor temperature warmer than outside.*”. This request requires retrieving the outdoor temperature and adjusting the indoor temperature accordingly. The Search Engine agent first retrieves the temperature, after which the Thermostat agent uses the result to determine the final temperature setting.

Partially Successful Case. The test case `simple_05` case with the user request “*Play anything for 1 hour.*”. This request triggers music playback and a timer request to the Clock agent. However, the vague instruction “play anything” should also prompt the system to request music recommendations from the Search Engine. Since this step is missed, the content selection remains underspecified.

Failed Case. For this simple_01 case, the request “*Help me find an Italian pasta recipe and see what ingredients I’m missing.*” requires retrieving a recipe and checking available ingredients. The system retrieves recipe suggestions but fails to coordinate with the Fridge agent to verify ingredient availability, resulting in incomplete execution.

These examples reveal two main failure patterns: (1) incomplete intent decomposition at the planning stage and (2) missed or insufficient collaboration at the device level. While the bounded single-step collaboration design ensures execution stability, it also limits the system’s ability to resolve complex multi-stage dependencies.

6 Conclusions

This work investigated whether LLM-based agents can effectively coordinate in smart home environments through flexible orchestration and address the limitations of existing approaches. To this end, we proposed an LLM-based multi-agent system that combines centralized task planning with dynamic peer-to-peer device collaboration. To address the challenge of lacking of benchmarking, we further develop an evaluation benchmark for LLM-based multi-agent smart home systems with varying levels of intent complexity. We conducted a comprehensive experimental evaluation of the smart home system through benchmark testing. Experimental results indicate that the system performs well in tasks of low to medium complexity, but its performance degrades in highly complex scenarios involving intricate intentions and a large number of devices. These findings suggest that coordination across a vast array of intentions and devices remains a significant challenge in LLM-based multi-agent smart home systems.

References

1. Acharya, D.B., Kuppan, K., Divya, B.: Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey. *IEEE Access* **13**, 18912–18936 (2025)
2. Ali, S., Yusuf, Z.: Mapping the smart-home market. Boston: The Boston Consulting Group (2018)
3. Bhat, A., Mondal, A., Tripathy, A.: Llm agents for internet of things (iot) applications (2025)
4. Chen, S., et al.: A survey on llm-based multi-agent system: Recent advances and new frontiers in application. *arXiv:2412.17481* (2024)
5. Ferrag, M.A., Tihanyi, N., Debbah, M.: From llm reasoning to autonomous ai agents: A comprehensive review. *arXiv preprint arXiv:2504.19678* (2025)
6. Groppe, J., Marquet, A., Walz, A., Groppe, S.: Automated archival descriptions with federated intelligence of llms. In: *International Conference on Database and Expert Systems Applications*. pp. 53–67. Springer (2025)
7. Halba, K., Griffor, E., Lbath, A., Dahbura, A.: Iot capabilities composition and decomposition: A systematic review. *IEEE Access* **11**, 29959–30007 (2023)
8. He, J., Treude, C., Lo, D.: Llm-based multi-agent systems for software engineering: Literature review, vision, and the road ahead. *ACM Transactions on Software Engineering and Methodology* **34**(5), 1–30 (2025)

9. Kessel, A.L., Sahri, S., Groppe, S., Groppe, J., Khorashadizadeh, H., Pignal, M., Pimparé, E.P., Vignes-Lebbe, R.: Impact of chatbots on user experience and data quality on citizen science platforms. *Computers* **14**(1) (2025), <https://doi.org/10.3390/computers14010021>
10. King, E., Yu, H., Lee, S., Julien, C.: Sasha: creative goal-oriented reasoning in smart homes with large language models. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* **8**(1), 1–38 (2024)
11. Li, X., Wang, S., Zeng, S., Wu, Y., Yang, Y.: A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth* **1**(1), 9 (2024)
12. Liu, X., Yu, H., Zhang, H., et al.: Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688* (2023)
13. Madakam, S.: Internet of things: smart things. *International journal of future computer and communication* **4**(4), 250 (2015)
14. Mohammadi, M., Li, Y., Lo, J., Yip, W.: Evaluation and benchmarking of llm agents: A survey. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. pp. 6129–6139 (2025)
15. Riquebourg, V., Menga, D., Durand, D., Marhic, B., Delahoche, L., Loge, C.: The smart home concept: our immediate future. In: *2006 1st IEEE international conference on e-learning in industrial electronics*. pp. 23–28. IEEE (2006)
16. Rivkin, D., Hogan, F., et al.: Sage: Smart home agent with grounded execution. *arXiv:2311.00772* (2023)
17. Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., et al.: A survey on large language model based autonomous agents. *Frontiers of Computer Science* **18**(6), 186345 (2024)
18. Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., et al.: The rise and potential of large language model based agents: A survey. *Science China Information Sciences* **68**(2), 121101 (2025)
19. Yang, H., Lee, W., Lee, H.: Iot smart home adoption: the importance of proper level automation. *Journal of Sensors* **2018**(1), 6464036 (2018)
20. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y.: React: Synergizing reasoning and acting in language models. In: *The eleventh international conference on learning representations* (2022)
21. Yehudai, A., Eden, L., Li, A., Uziel, G., Zhao, Y., Bar-Haim, R., Cohan, A., Shmueli-Scheuer, M.: Survey on evaluation of llm-based agents. *arXiv preprint arXiv:2503.16416* (2025)
22. Zhang, X., Zhang, C., Sun, J., Xiao, J., Yang, Y., Luo, Y.: Eduplanner: Llm-based multi-agent systems for customized and intelligent instructional design. *IEEE Transactions on Learning Technologies* (2025)
23. Zhu, K., Du, H., Hong, Z., Yang, X., Guo, S., Wang, D.Z., Wang, Z., Qian, C., Tang, R., Ji, H., et al.: Multiagentbench: Evaluating the collaboration and competition of llm agents. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. pp. 8580–8622 (2025)
24. Çalkıylmaz, U., Nayak, N., Groppe, J., Groppe, S.: Optima: A transaction-based framework with throughput optimization for very complex multi-agent systems. *arXiv* **2511.03761** (2025), <https://doi.org/10.48550/arxiv.2511.03761>